

Academia de Studii Economice din București

Facultatea de Cibernetică, Statistică și Informatică Economică

PROIECT: REȚELE DE CALCULATOARE

Rezolvarea adresei IP dupa numele calculatorului

Covrig Eduard-Gabriel - client

Constantin Arthur-Stefan - server

Ciobanu Cătălin-Marian - server

1. Introducere și Arhitectură

Proiectul reprezintă un sistem distribuit client-server care simulează funcționarea unui resolver DNS (Domain Name System). Arhitectura este compusă din trei entități principale, comunicarea realizându-se concurent prin socket-uri TCP, cu date formate JSON:

Clientul: Reprezintă interfața de interogare. Menține un fișier local de cache (`client_cache.txt`) pentru a memora rezultatele anterioare.

Serverul Principal (Port 8702/8712/8719, Dockerizat): Gestionează domeniile `.principal.ro`. Deține propria bază de date (`server_db_87XX.txt`). Dacă nu cunoaște un domeniu, direcționează cererea mai departe.

Serverul Secundar (Port 8797/8798/8799) Gestionează domeniile `.secundar.ro`. Primește cereri de la serverul principal și răspunde din propria bază de date (`server_db_87XX.txt`).

2. Funcționalitățile de baza

Rezolvare corectă din fișier local: La fiecare rulare, clientul încarcă `client_cache.txt` în memorie. La orice interogare, verifică mai întâi acest dicționar și răspunde instantaneu.

Forward corect către server responsabil: Serverul principal are definit un dicționar de FORWARDING. Când primește o cerere ce se termină în `.secundar.ro`, deschide un nou socket TCP către portul 8797/8798/8799, obține IP-ul și îl trimite înapoi clientului inițial.

Cache corect pozitiv și negativ: Clientul memorează atât IP-urile găsite, cât și răspunsurile de tip `NOT_FOUND`. Mai mult, s-a implementat Server-Side Caching: serverul memorează și el la rândul său ce învață de la serverul secundar.

3. Demonstrarea funcționalităților

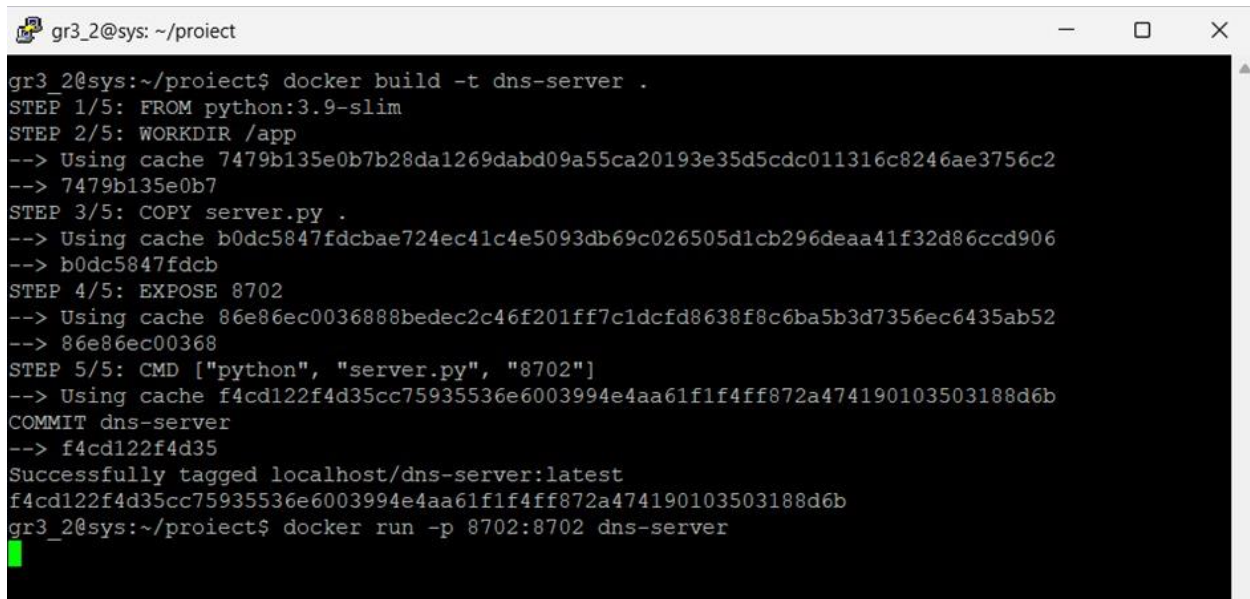
1. Pornire server principal în Docker (Ciobanu Catalin-Marian)

Comanda: (`docker compose up --build`)

```
docker build -t dns-server .
```

```
docker run -p 8702:8702 dns-server
```

Mesajul care confirmă pornirea serverului în Docker:

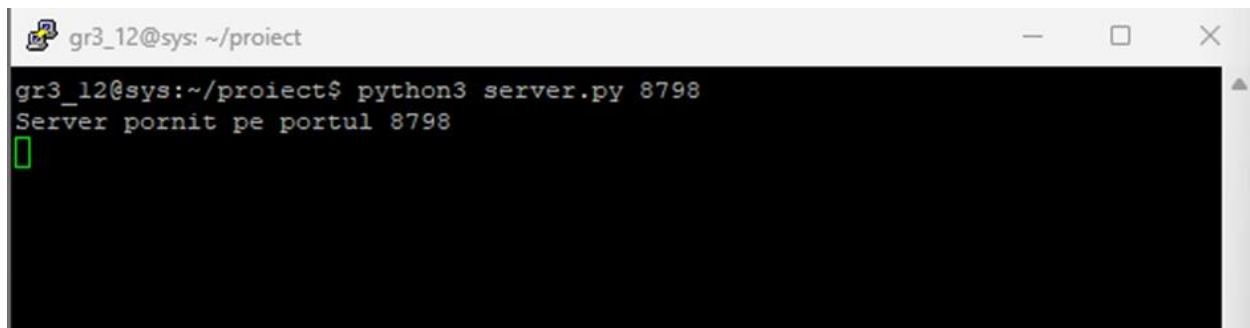
A terminal window titled 'gr3_2@sys: ~/proiect' showing the process of building and running a Docker container. The user runs 'docker build -t dns-server .' which shows five steps: FROM python:3.9-slim, WORKDIR /app, COPY server.py, EXPOSE 8702, and CMD ["python", "server.py", "8702"]. The build is successful, tagging the image as localhost/dns-server:latest. Then, the user runs 'docker run -p 8702:8702 dns-server' and a green cursor is visible on the next line.

```
gr3_2@sys:~/proiect$ docker build -t dns-server .
STEP 1/5: FROM python:3.9-slim
STEP 2/5: WORKDIR /app
--> Using cache 7479b135e0b7b28da1269dabd09a55ca20193e35d5cdc011316c8246ae3756c2
--> 7479b135e0b7
STEP 3/5: COPY server.py .
--> Using cache b0dc5847fdcb724ec41c4e5093db69c026505d1cb296deaa41f32d86ccd906
--> b0dc5847fdcb
STEP 4/5: EXPOSE 8702
--> Using cache 86e86ec0036888bedec2c46f201ff7c1dcfd8638f8c6ba5b3d7356ec6435ab52
--> 86e86ec00368
STEP 5/5: CMD ["python", "server.py", "8702"]
--> Using cache f4cd122f4d35cc75935536e6003994e4aa61f1f4ff872a474190103503188d6b
COMMIT dns-server
--> f4cd122f4d35
Successfully tagged localhost/dns-server:latest
f4cd122f4d35cc75935536e6003994e4aa61f1f4ff872a474190103503188d6b
gr3_2@sys:~/proiect$ docker run -p 8702:8702 dns-server
█
```

2. Pornire server secundar responsabil pentru alt domeniu (Constantin Arthur-Stefan)

Comanda: `python server.py 8798`

Mesajul de start: Server pornit pe portul 8798 și apariția automată a fișierului `server_db_8798.txt` în folder.

A terminal window titled 'gr3_12@sys: ~/proiect' showing the execution of the command 'python3 server.py 8798'. The output is 'Server pornit pe portul 8798' followed by a green cursor.

```
gr3_12@sys:~/proiect$ python3 server.py 8798
Server pornit pe portul 8798
█
```

```
gr3_12@sys: ~/proiect
GNU nano 7.2 server_db_8798.txt
pc1.secundar.ro=10.0.0.5
pc2.secundar.ro=10.0.0.6
pc3.secundar.ro=10.0.0.7
```

3. Pornire client (Covrig Eduard-Gabriel)

Comanda: `python3 client.py`

Promptul clientului care așteaptă date: `> Introdu numele de rezolvat:`

```
[gr3_19@sys:~/proiect$ python3 client.py
> Introdu numele de rezolvat: █
```

4. Interogare pentru un nume rezolvat local pe server principal (Covrig Eduard-Gabriel)

Comanda: `pc1.principal.ro`

Răspunsul preluat de pe rețea din baza principală: `[SERVER] pc1.principal.ro -> 192.168.1.10`

```
[gr3_19@sys:~/proiect$ python3 client.py
[> Introdu numele de rezolvat: pc1.principal.ro
[SERVER] pc1.principal.ro -> 192.168.1.10
> Introdu numele de rezolvat: █
```

5. Interogare pentru un nume rezolvat prin server secundar (forward) (Covrig Eduard-Gabriel)

Comanda: `pc1.secundar.ro`

Răspunsul adus din serverul secundar (Terminalul 2), prin forward: `[SERVER] pc1.secundar.ro -> 10.0.0.5`

```
[gr3_19@sys:~/proiect$ python3 client.py
[> Introdu numele de rezolvat: pc1.principal.ro
[SERVER] pc1.principal.ro -> 192.168.1.10
[> Introdu numele de rezolvat: pc1.secundar.ro
[SERVER] pc1.secundar.ro -> 10.0.0.5
> Introdu numele de rezolvat: █
```

6. Interogare repetată pentru același nume și rezolvare din cache (Covrig Eduard-Gabriel)

Comanda: pc1.principal.ro

Sistemul folosește memoria locală, nu mai apelează rețeaua:

```
[CACHE LOCAL] pc1.principal.ro -> 192.168.1.10
```

```
[CACHE LOCAL] pc1.secundar.ro -> 10.0.0.5
```

```
[gr3_19@sys:~/proiect$ python3 client.py
[> Introdu numele de rezolvat: pc1.principal.ro
[SERVER] pc1.principal.ro -> 192.168.1.10
[> Introdu numele de rezolvat: pc1.secundar.ro
[SERVER] pc1.secundar.ro -> 10.0.0.5
[> Introdu numele de rezolvat: pc1.principal.ro
[CACHE LOCAL] pc1.principal.ro -> 192.168.1.10
[> Introdu numele de rezolvat: pc1.secundar.ro
[CACHE LOCAL] pc1.secundar.ro -> 10.0.0.5
> Introdu numele de rezolvat: █
```

7. Interogare pentru un domeniu necunoscut și memorare rezultat negativ (Ciobanu Catalin-Marian0)

Căutăm un domeniu care nu există nicăieri

Comanda: pc99.fals.ro

Sistemul a căutat pe server și nu a găsit, răspunsul corect fiind eroarea asumată:

```
[SERVER] pc99.fals.ro -> NOT_FOUND
```

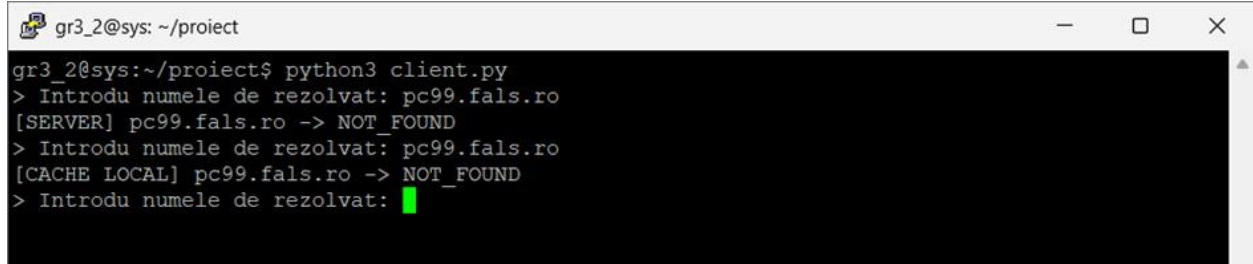
```
gr3_2@sys: ~/proiect
gr3_2@sys:~/proiect$ python3 client.py
> Introdu numele de rezolvat: pc99.fals.ro
[SERVER] pc99.fals.ro -> NOT_FOUND
> Introdu numele de rezolvat: pc99.fals.ro
[CACHE LOCAL] pc99.fals.ro -> NOT_FOUND
> Introdu numele de rezolvat: █
```

8. Reinterogare pentru același domeniu și răspuns imediat din cache negativ (Ciobanu Catalin-Marian)

Căutăm din nou același domeniu greșit de la pasul anterior: `pc99.fals.ro`

Sistemul a reținut eșecul și returnează eroarea din memoria locală pentru a economisi resurse:

```
[CACHE LOCAL] pc99.fals.ro -> NOT_FOUND
```

A terminal window titled 'gr3_2@sys: ~/proiect' with standard window controls. The terminal shows the execution of a Python script 'client.py'. The user enters 'pc99.fals.ro' twice. The first time, the output is '[SERVER] pc99.fals.ro -> NOT_FOUND'. The second time, the output is '[CACHE LOCAL] pc99.fals.ro -> NOT_FOUND'. The prompt is followed by a green cursor.

```
gr3_2@sys:~/proiect$ python3 client.py
> Introdu numele de rezolvat: pc99.fals.ro
[SERVER] pc99.fals.ro -> NOT_FOUND
> Introdu numele de rezolvat: pc99.fals.ro
[CACHE LOCAL] pc99.fals.ro -> NOT_FOUND
> Introdu numele de rezolvat: █
```